Lola Retro

A CRT shader for Unity. User manual, version 1.12.0

Contents

1. Overview
2. Install in a minute
3. Put it on your game
4. Requirements
5. The console's picture
6. Presets
7. From a script
8. Good to know
9. HDR displays
10. XR
11. Web (WebGL 2)
12. Performance
13. Tested
14. License

1. Overview

Lola Retro as a Unity package: a CRT shader that simulates the whole analog chain, not just scanlines.

```
console video encoder → cable → TV decoder → electron beam → phosphor mask → glass
```

Your game goes through the same passes as in the RetroArch version, with the same presets and every parameter: composite blends dithering into new colours and makes 1-pixel stripes shimmer, RGB keeps them sharp, the beam grows with brightness, the mask and the glass are drawn at your screen's resolution. Made for indie games with a retro look: pixel art, PS1-style 3D, anything that should look like it ran on a TV or monitor of the 80s and 90s.



Dracula from Castlevania: Symphony of the Night, the same frame in every picture. From left: the raw pixels; a photo of a real Sony KV-13M51 Trinitron on composite, by CRTpixels; Lola Retro with the Trinitron preset switched to composite, and with the consumer TV composite preset. The Unity version draws the same picture as the RetroArch version it is made from (see Tested).

2. Install in a minute

1. On Lola Retro's Asset Store page, press **Add to My Assets** (it is free).
2. In Unity: **Window → Package Manager** (in newer versions **Window → Package Management → Package Manager**), then **My Assets → Lola Retro → Download → Import**, and **Import** once more in the window that opens.
3. To see it at once: open the scene `Assets/Lola Retro/Demo/Lola Retro Demo` and press **Play**. The arrow keys change the TV, **C** the cable.

3. Put it on your game

1. In the **Hierarchy**, click the camera that shows your game (usually *Main Camera*).
2. In the **Inspector**: **Add Component**, type `Lola Retro`, press **Enter**.
3. If the component shows a box with a **Fix it** button (in URP and HDRP it does the first time), press it.
4. Pick a **Preset** and press **Play**.

That is all. Your game now shows on the TV most people had: a consumer TV on the composite cable. The presets under *Systems* are the TV or monitor most people had for that console (the NES and SNES dots, the Mega Drive's rainbow), and every value under **Settings** can be changed while the game runs.

Lola Retro makes a console picture of 240 lines from what the camera sees (**Lines**; the width follows the camera's shape). For pixel art, set Lines to your game's height, or see The console's picture.

Without the Asset Store, from GitHub: **Window → Package Manager → + → Install package from git URL...** and enter `https://github.com/zoran123456/lola-retro.git?path=/unity/lola-retro` (add `#v1.12` or another release tag at the end to stay on that version). The Demo is then in the Package Manager, under **Lola Retro → Samples → Demo → Import**. Use one or the other, not both in the same project.

What Fix it does

- **URP** draws Lola Retro through its renderer feature: Fix it adds **Lola Retro** to the camera's renderer asset (the Universal Renderer Data, or the Renderer 2D Data in a 2D project), as **Add Renderer Feature → Lola Retro** would.
- **HDRP** runs it as a custom post-process: Fix it adds `LolaRetro.LolaRetroVolume` to **Project Settings → Graphics → HDRP Global Settings → Custom Post Process Orders → After Post Process**. No Volume is needed. A second, milder note sets HDRP's post-processing buffer to R16G16B16A16 (its default keeps only 6 or 7 bits a channel, and Lola Retro's picture would lose about one step of 8-bit colour).
- **Built-in render pipeline:** nothing to fix.

4. Requirements

- Unity 6 (6000.0) or newer.
- URP (with Render Graph, the default in Unity 6), HDRP or the Built-in render pipeline.
- Desktop (Direct3D 11/12, Metal, Vulkan, OpenGL Core) or the web (WebGL 2). It needs about what the RetroArch version needs, see Performance. Phones have not been tried.
- SDR and HDR displays, and XR headsets (URP and HDRP).

5. The console's picture

A CRT shows a picture of a few hundred lines, and the passes need it at that resolution: the encoder works on the console's pixels, the beam draws one line per console line. There are two ways to give it one.

From the camera's image (no Input). The camera renders at the screen's resolution as usual, and each console pixel is the average of what it covers: like a 3D game rendered at 320×240, with clean edges. Set **Lines** (224 or 240 for most 8- and 16-bit looks, 480 for an interlaced picture) and leave **Width** at 0 for square pixels. For pixel art, set Lines to your game's height: with the

camera drawing its pixels at a whole multiple of it (the Pixel Perfect Camera without *Upscale Render Texture*), every console pixel is exactly one of your pixels.

From a render texture (Input). For pixel-exact games, or PS1-style 3D at a low resolution: one camera renders the game into a render texture at the console's resolution (for example 320×240, Filter Mode Point), and the camera that shows the game has Lola Retro with that texture as **Input**. Every texel is then one console pixel. That second camera renders nothing itself: Culling Mask *Nothing*, Background *Solid Color*, black. It must render after the first one (a higher **Priority** in URP, **Depth** in the Built-in pipeline).

The **Tube shape** is 4:3 for console pictures up to 1.5:1 and the picture's own shape for wider ones (a 16:9 game gets a widescreen TV), in the middle of the screen with black around it. *Fill the screen* stretches the tube to the screen. **Around the tube** (other than Black) always fills the screen and draws a 4:3 tube in it.

6. Presets

Preset	What it is
Consumer TV, composite	Start here. The TV most people had, on the composite cable that came in the box.
Consumer TV, SCART RGB	The same TV on RGB: a clean picture.
TV show (broadcast)	Games as they looked on a TV programme about games: through a studio, onto tape, through the antenna.
Sony Trinitron, S-Video	Aperture grille, flat vertically, cool white. PS2, GameCube, N64.
Studio monitor, RGB	The PVM look: sharp, a fine grille, calibrated colour.
Sony PVM-14L2, composite	A real PVM on composite, with its comb filter.
Old TV, RF	The antenna cable: snow, ghosting, soft focus, a round tube.
CRT projector	Three tubes on a flat screen, like a deluxe arcade cabinet.
PC monitor (VGA), good, premium, premium+, extreme	A mid-90s 15-inch PC monitor described in millimetres, and better ones.
Commodore 1084S (Amiga)	The Amiga monitor, from its service manual.
Systems	One preset per console or computer, as in the RetroArch version's `systems` folder.

Flat screen makes any of them flat with square corners. A preset sets every value under **Settings**; change them after that as you like (choosing a preset again resets them). They are the parameters of the RetroArch version and mean the same, see its Parameters.

7. From a script

```
using LolaRetro;

var crt = Camera.main.GetComponent<LolaRetroEffect>();
crt.ApplyPreset(LolaRetroPreset.OldTVRF);        // every value of the preset
crt.settings.cable = Cable.RGB;                  // then any parameter
crt.settings.roundedCorners = 0f;
crt.enabled = false;                             // off
```

An options menu can offer the cable (RF, composite, S-Video, RGB) and the presets; the picture changes in the next frame.

Everything is in the namespace `LolaRetro`:

Name	What it is
`LolaRetroEffect`	The component on the camera. Its fields are the Inspector's: `preset`, `flatScreen`, `input`, `lines`, `width`, `tubeShape`, `settings`.
`LolaRetroEffect.ApplyPreset(preset, flat = false)`	Sets every value of `settings` to the preset's (and `preset`, `flatScreen`).
`LolaRetroEffect.IsRendering`	True while Lola Retro draws this camera's picture (false when the render pipeline still needs Fix it).
`LolaRetroSettings`	Every parameter, one field each, with the Inspector's names (`cable`, `mask`, `maskStrength`, `aroundTheTube`, …); the tooltips say what they do.
`LolaRetroPreset`	The presets, as in the Inspector's Preset list.
`Cable`, `MaskType`, `TubeShape`, …	The choices of the fields that have a list.
`LolaRetroFeature` (URP), `LolaRetroVolume` (HDRP)	What draws it in those pipelines; Fix it sets them up.

8. Good to know

- **UI.** A Screen Space - Overlay canvas is drawn after Lola Retro, sharp on top of the tube. To have the UI on the tube too, render it with the game: Screen Space - Camera on the camera whose image Lola Retro shows.
- **URP's own anti-aliasing and upscaling** run after Lola Retro and would blur the mask: on the camera, set Post-processing *Anti-aliasing* to None, and keep the URP asset's *Render Scale* at 1. URP's *Dithering* is not needed: the last pass dithers like the RetroArch version. In HDRP the same: the camera's *Anti-aliasing* None, *Dithering* off.
- **Colour.** Unity renders in linear light, a console sent gamma-coded RGB: the first pass codes the picture as an emulator's frame is coded, the last one turns the result back. Post-processing (tonemapping, colour grading) happens before Lola Retro, as if it were part of the game.
- **Afterglow** (Phosphor afterglow) works as in RetroArch: each camera keeps its last frame.
- **A TV in your game's world.** A camera with Lola Retro that renders into a render texture

(*Output Texture* in URP and HDRP, *Target Texture* in the Built-in pipeline) gives the CRT's picture as a texture: put it on the screen of a TV or monitor model.

9. HDR displays

With HDR output on (**Player Settings → Allow HDR Display Output**, and *HDR* in the URP asset), Lola Retro works like the RetroArch version's `hdr` presets: a full white field is shown at paper white, and the phosphor dots and line centres, brighter than that average as on a real tube, use the display's headroom up to its peak, instead of being squeezed under the SDR white level. Paper white and the peak are the display's own, as Unity reads them; the *Tonemapping* override's *Paper White* and *Max Nits* change them when their *Detect* boxes are off. Unity codes the picture for the display itself (HDR10 or scRGB).

With a bright display (peak about 4× paper white or more) you can raise **Mask strength** towards 1.0: the gaps between the phosphors go dark like on a real tube and the picture stays as bright.

10. XR

In URP and HDRP, single-pass and multi-pass, each eye gets its own tube, from its own image. The tube is drawn in each eye's view, like a screen in front of your eyes; for a TV standing in the world, use a camera that renders into a render texture (see *A TV in your game's world*).

11. Web (WebGL 2)

Unity's web builds use WebGL 2. The passes need float render targets (`EXT_color_buffer_float`, which every desktop browser has) and, for the beam pass, textures 8192 wide. Where the GPU's limit is smaller (4096 on many phones) the beam pass is that wide; the picture changes by about 1% of one 8-bit step on average.

12. Performance

The chain has nine passes (ten on a TV show) and the encoder and decoder do real signal filtering, as in the RetroArch version: it needs a reasonably modern GPU; integrated graphics from the last few years should be fine at 1080p. The cost hardly depends on the game: the encoder and decoder work at 1280 samples per line whatever the console picture's width, the beam pass once per console line, the rest once per screen pixel. In a browser on an Apple M4 Pro (WebGL 2) the whole chain takes 2 to 3 ms per frame at 1080p to 1440p, and 4.6 ms at 4K.

13. Tested

In Unity 6.6 (6000.6.7f1) on a Mac: the package compiles without errors or warnings, and the Demo's picture (the SNES preset on composite) is RetroArch's in the Built-in render pipeline, in URP and in HDRP (with its post-processing buffer at R16G16B16A16) on Metal, and in a web build (WebGL 2) for the Built-in pipeline and URP: the pictures agree to within one step of 8-bit colour,

apart from a few single pixels per frame where the mask changes colour. The Fix it buttons were run there too.

The passes are made from the RetroArch shaders by a converter (`tools/make_unity.py` in the repository). Every preset, cable and option was also checked by running both on the same GPU, with the Unity passes run the way the package runs them, and with the rows the right way up for both kinds of graphics API (`tools/unity_preview.py`). The same holds for the HDR passes against the RetroArch version's `hdr` presets (within one step of 10-bit HDR10, which RetroArch dithers by half a step), for an eye's image read from a texture array (XR, HDRP), and in WebGL 2 in a browser (`tools/webgl_preview.py`).

On OpenGL ES and WebGL the shader uses GLSL made from the same passes by SPIRV-Cross. Unity's own translation there keeps integers in float registers as tiny bit patterns that these GPUs flush to zero, and the picture came out black.

Not yet run in Unity: HDR output and XR. Reports are welcome.

14. License

MIT, see the LICENSE file in this folder.